0.0.1 Ultrasonic Hardware Overview

The ultrasonic sensors are arranged as a distributed array around the rover. Each sensor is powered from the 5 V rail and produces a 5 V digital echo pulse in response to a short trigger pulse. The overall electrical organization of the array is shown in Figure 1.

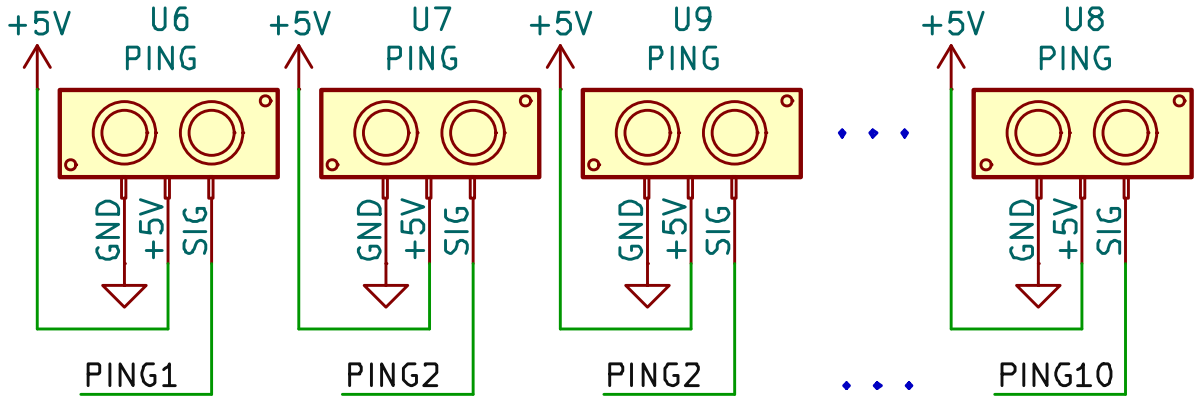

Figure 1: Schematic excerpt of the ultrasonic sensor array. Ten sensors are powered from the 5 V rail and form a distributed ranging array for the KIRB-3 platform.

The trigger/echo interface of each ultrasonic module is a driven, bidirectional line, making it unsuitable for direct multi-sensor wiring or direct connection to 3.3 V MCU inputs. To implement a safe shared PING Bus, the design employs two stages:

- **An analog multiplexer** that connects exactly one sensor at a time to the shared line (the *PING Bus*), preventing cross-driving between sensors.
- **A bidirectional buffer with direction control** that allows the MSPM0 to drive the trigger pulse, then switches to receive mode to pass the sensor's 5 V echo while level-protecting the 3.3 V domain.

This architecture does more than reduce pin count: it enables *every* sensor to use the MSPM0's hardware capture unit to measure echo pulse width. Without multiplexing, the limited number of timer capture channels would restrict how many sensors could be measured using precise hardware timing.

The routing and level-conditioning circuitry is shown in Figure 2.

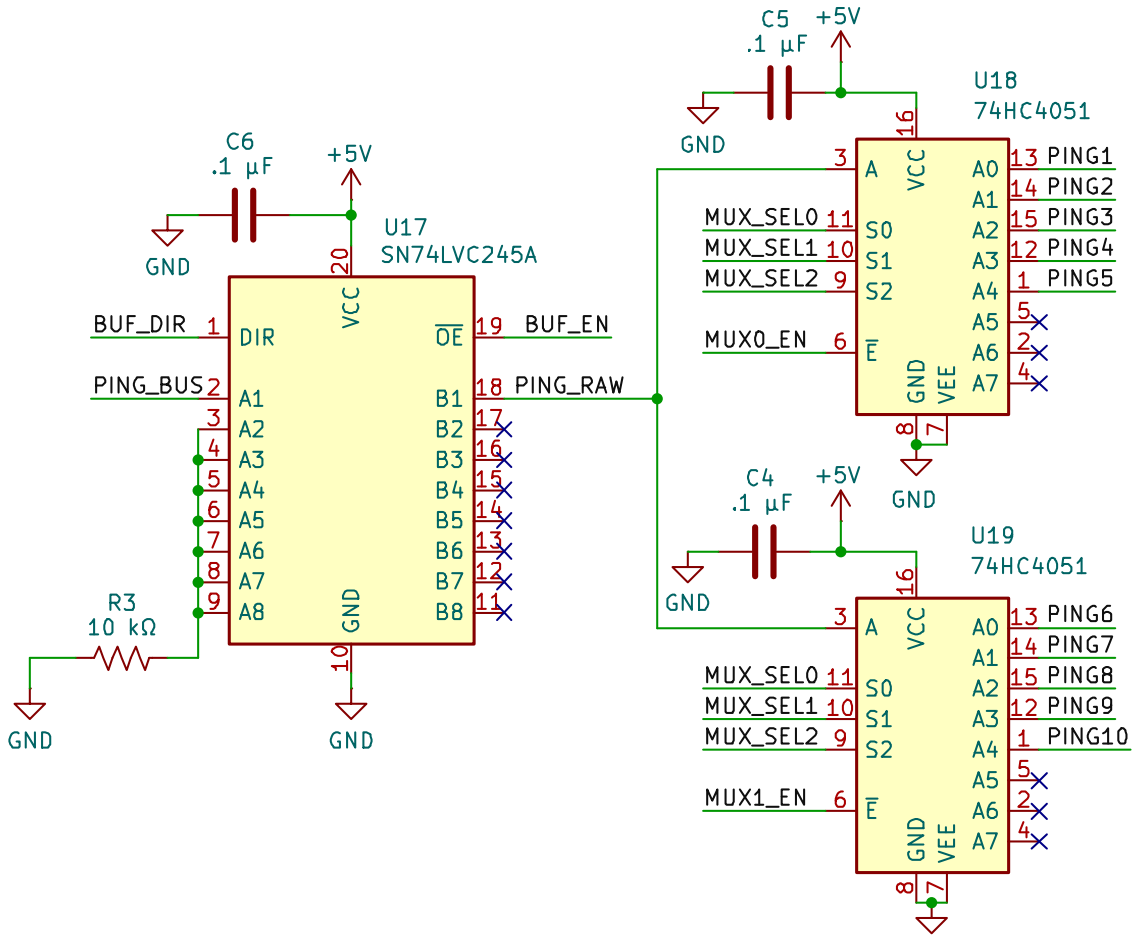

Figure 2: Schematic excerpt of the ultrasonic routing subsystem. A multiplexer selects one sensor onto the shared PING Bus, while the bidirectional buffer provides direction control and level protection between the sensor array and the MSPM0's 3.3 V domain.

To service all ten sensors over a single PING Bus, the firmware runs a simple round-robin scheduler over the sensor index `sensor_idx`. On each period of the Ultrasonic Schedule Timer, it selects the next sensor, performs exactly one measurement on that channel, and then advances the index with wrap-around. The high-level behavior of this scheduler is shown in Figure 3.

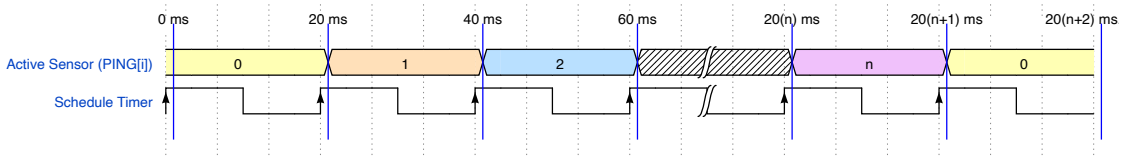

Figure 3: Round-robin ultrasonic scheduling. Each period of the schedule timer selects one sensor onto the PING Bus and triggers a single measurement, then advances the active sensor index.

0.0.2 Single-Cycle Trigger and Echo Timing

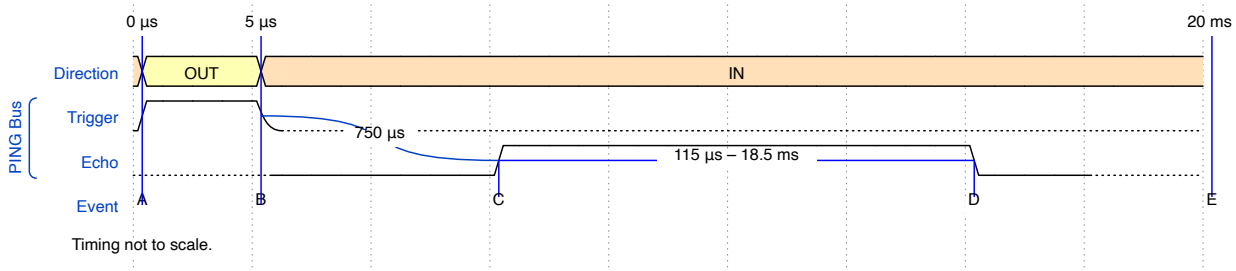

Figure 4: Timing of one ultrasonic measurement on the PING Bus. The schedule timer, echo timer, buffer direction, and bus state progress through one complete trigger/echo cycle for the selected sensor.

The key stages are: placing the PING Bus in a safe state, selecting the active sensor, issuing and ending the trigger pulse, arming the echo timer on the first edge, and capturing the pulse width on the falling edge.

0.0.3 Interrupt-Driven Control Flow

These timing events map directly to interrupt service routines (IRQs) on the schedule timer and echo timer. The flowchart in Figure 5 summarizes the firmware logic that executes one measurement per schedule period and corresponds directly to the timing shown in Figure 4.

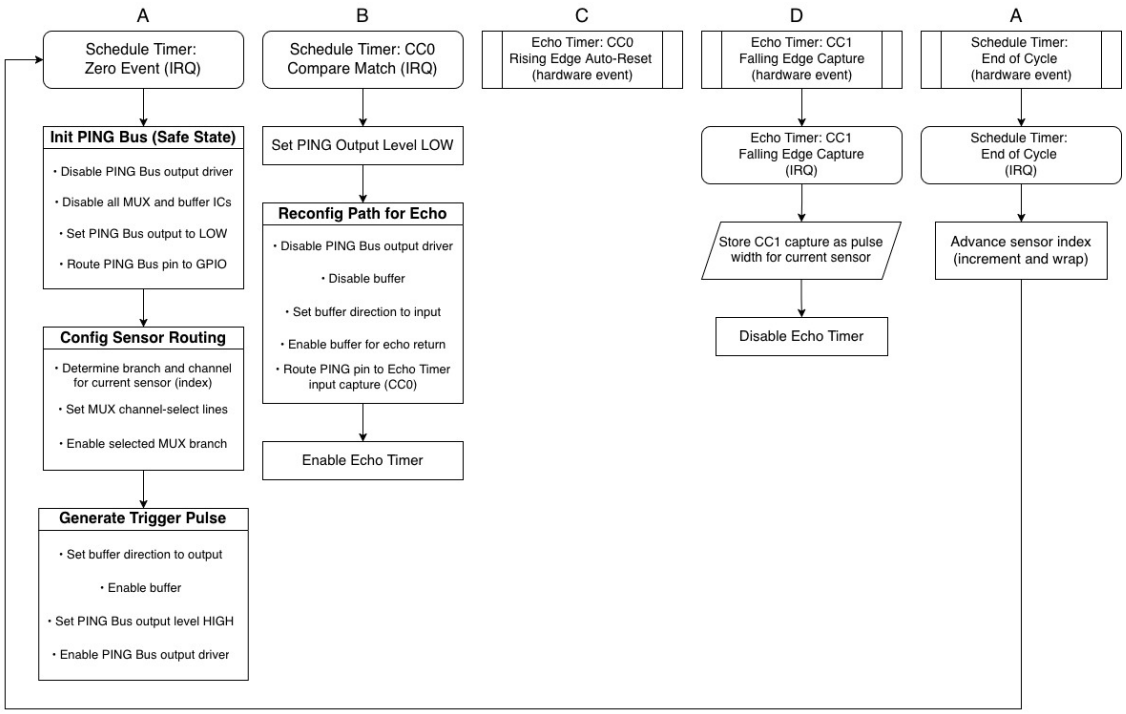

Figure 5: Firmware control flow for one ultrasonic measurement cycle, driven by schedule timer interrupts and echo timer hardware events.

Together, the timing diagram and flowchart describe how KIRB-3 multiplexes its ultrasonic sensors onto the shared PING Bus and uses MSPM0 timers to perform precise ranging across the sensor array.

Differential-Drive Odometry

Odometry refers to estimating a robot's position (x, y) and orientation θ over time by integrating wheel encoder measurements. In a differential-drive system, changes in left and right wheel displacements determine the robot's forward motion and rotation.

Symbol	Name / Role	Definition / Notes
$N_L,\ N_R$	Encoder counts (cum.)	Cumulative quadrature counts (ticks) for left/right wheels.
$\Delta N_L,\ \Delta N_R$	Encoder count increments	Counts accumulated since the last update step (ticks).
T	Ticks per revolution	Encoder ticks per wheel revolution (include quadrature mode and gearbox).
R	Wheel radius	In meters. Used to convert ticks $\rightarrow$ distance.
b	Wheel track	Lateral distance between wheel contact patches (m).
$(x,\ y,\ \theta)$	Robot pose	World-frame position and heading; θ measured CCW from $+x$ (radians).
$\Delta s_L,\ \Delta s_R$	Wheel travel (step)	Left/right wheel arc lengths over the last step: $\Delta s_L = \frac{2\pi R}{T}\,\Delta N_L, \quad \Delta s_R = \frac{2\pi R}{T}\,\Delta N_R$.
Δs	Chassis forward travel (step)	Body-frame arc length over the last step: $\Delta s = \frac{1}{2}(\Delta s_R + \Delta s_L)$.
$\Delta\theta$	Heading change (step)	Turn over the last step: $\Delta\theta = \frac{\Delta s_R - \Delta s_L}{b}$.
s	Path length (cumulative)	Total forward distance traveled along the chassis arc. Update each step by $s \leftarrow s + \Delta s$. (Some notes use s for a single step; here s is cumulative and Δs is per step.)

1. **Wheel travel.** Converts encoder ticks into physical distance for each wheel. Each tick represents a small rotation, and multiplying by $\frac{2\pi R}{T}$ gives the linear displacement:

$$\Delta s_L = \frac{2\pi R}{T}\,\Delta N_L, \qquad \Delta s_R = \frac{2\pi R}{T}\,\Delta N_R$$

2. **Differential motion.** Computes how far the robot moved forward and how much it rotated. The average wheel travel gives forward motion, while the difference gives rotation about the centerline:

$$\Delta s = \frac{\Delta s_R + \Delta s_L}{2}, \qquad \Delta\theta = \frac{\Delta s_R - \Delta s_L}{b}$$

3. **Pose update (midpoint form).** Updates global position (x, y) and heading θ using the arc traced during motion. The midpoint form accounts for the slight curvature of the path:

$$x \leftarrow x + \Delta s\cos\!\left(\theta + \tfrac{\Delta\theta}{2}\right), \quad y \leftarrow y + \Delta s\sin\!\left(\theta + \tfrac{\Delta\theta}{2}\right), \quad \theta \leftarrow \mathrm{wrap}(\theta + \Delta\theta)$$

   This effectively "walks" the pose forward by one small motion step.

4. **Exact arc formulation (optional).** When the robot turns significantly, you can compute the exact circular arc rather than the midpoint approximation. The instantaneous center of curvature (ICC) radius is:

$$R_{\text{icc}} = \frac{\Delta s}{\Delta\theta} = \frac{b}{2}\,\frac{\Delta s_R + \Delta s_L}{\Delta s_R - \Delta s_L},$$

and the pose update follows the true circular trajectory:

$$x \leftarrow x + R_{\text{icc}}\,[\sin(\theta+\Delta\theta) - \sin\theta], \quad y \leftarrow y - R_{\text{icc}}\,[\cos(\theta+\Delta\theta) - \cos\theta], \quad \theta \leftarrow \text{wrap}(\theta+\Delta\theta)$$

If $\Delta\theta \approx 0$ (straight motion), this simplifies to

$$x \leftarrow x + \Delta s\cos\theta, \qquad y \leftarrow y + \Delta s\sin\theta.$$

5. **Velocities (fixed timestep Δt).** Converts incremental motion into linear and angular velocities. These describe the robot's instantaneous motion:

$$v_L = \frac{2\pi R}{T}\frac{\Delta N_L}{\Delta t}, \qquad v_R = \frac{2\pi R}{T}\frac{\Delta N_R}{\Delta t}, \qquad v = \frac{v_R + v_L}{2}, \quad \omega = \frac{v_R - v_L}{b}$$

Integrating these gives continuous-time motion:

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega.$$

Notes.

1. Ensure encoder signs are consistent so both wheels forward give $\Delta\theta \approx 0$.
2. Include gearbox ratio and quadrature multiplication in T.
3. Measure b between effective wheel contact points; small errors bias θ.
4. Odometry drifts over time; fuse with IMU/vision for long runs.